Lambda World speaker script

4,103 spoken words. Planning pace: 125 words/minute, plus 04:20 of pauses and visual holds. Speech and holds: 37:09; with 02:00 for questions: 39:09. This is an estimate, not an aloud rehearsal.

Speak the prose; square-bracketed stage cues in exports are not spoken. Long numerical traces have one continuous passage plus a final-state passage. No Clojure or OR prerequisite; explain notation at first use. Timing is a calculation, not an aloud rehearsal.

01 Rebuilding Operations Research in Higher-Order Functions

00:00–00:15 · 25 words · 3s visual holds

Phase 1

> Let the title settle.

Hello, I'm Jiri. We'll build three operations research examples with higher-order functions, watch them run, and then examine a proof about one of their updates.

02 About me

00:15–00:41 · 48 words · 3s visual holds

Phase 1

> Point briefly to the three lines; do not read them twice.

I've been developing software all my life. My education is in economics, so I'm interested in how we make decisions when resources are limited. Thirteen years ago, I started the Barcelona Meetup Group. This talk brings together those interests: functional programming, mathematical models, and decisions we can explain.

03 Why Clojure?

00:41–01:09 · 51 words · 3s visual holds

Phase 1

> Move down the four reasons.

I chose Clojure for small functions, immutable data, and the REPL. Keeping old states makes experiments easy to compare. Code is also data, which we'll use later when we get to proofs. You don't need Clojure experience today. I'll introduce its notation where it matters; the functional ideas should be familiar.

04 Operations research

01:09–01:46 · 72 words · 3s visual holds

Phase 1

> Leave the concrete examples for later.

Operations research uses models to compare choices under objectives and constraints. Optimization searches for a plan. Decision-making under uncertainty compares possible outcomes.

I also include validation: checking whether a system respects its specification. Reachability usually belongs to verification; it matters here because a profitable decision system can still violate a requirement. These are different questions. We'll use a different kind of state for each, but the same way of building an iteration.

05 Functions in. Functions out.

01:46–03:15 · 167 words · 9s visual holds

Phase 1

> Point to the returned fn, then the captured n.

Let’s start with something small: improving a guess for a square root. In Clojure a call puts the function first, then its arguments. Defn names a function; square brackets list its parameters. This function takes the number whose root we want, and returns an update function. That returned function averages the guess with the number divided by the guess. We use a positive number and a positive starting guess here, so division by zero is avoided. The outer function chooses the problem; the inner function performs one update.

Phase 2

> Right. Read the three independent results.

Here, map applies that update separately to three guesses. Returning this function makes the constructor higher-order. Map is higher-order because it takes the returned function as an argument. Each input gets one update. There is no connection between the three results. In Clojure, fn introduces an anonymous function, just as a lambda would elsewhere.

Phase 3

> Right. Point from one result back to the input.

But to solve the problem, I want the result of one update to become the input to the next. That is what iterate gives me.

06 iterate: state → state → …

03:15–04:17 · 98 words · 15s visual holds

Phase 1

> Point to state 0.

The first element of iterate is the starting value itself: one. No update has happened yet.

Phase 2

> Right once.

One update gives one point five. We keep it as the next state.

Phase 3

> Right once.

Now the function receives one point five, rather than the original one. The new guess is about one point four one seven.

Phase 4

> Right once.

The next guess is closer again. Each call has exactly the same interface: a guess in, a guess out.

Phase 5

> Right once. Pause on the complete chain.

Take asks for five states from this lazy sequence. Iterate itself has no idea what a square root is, or when we should stop. We supply that knowledge.

07 A fixed point stays the same

04:17–04:55 · 66 words · 6s visual holds

Phase 1

> Point out that this example switches to the square root of FOUR.

For an exact example, let’s ask for the square root of four. Starting at three, the update gives about two point one six seven. The value changes, so we could keep going.

Phase 2

> Right. Point to 2 on both sides.

Starting at two, the update returns two. Two is a fixed point of this particular function: applying the function leaves it unchanged. That definition does not promise that every starting point will reach it.

08 step + converged?

04:55–06:24 · 166 words · 9s visual holds

Phase 1

> Point to the parameters and iterate, not every implementation line.

Now we separate two jobs. Step tells us how to update a state. Converged tells us whether two successive states are close enough. The question mark is part of the predicate's name. This helper returns the trace, so we can inspect it. The function and stopping test are both arguments.

Phase 2–penultimate

> Advance through the guesses; do not read all decimal places. This paragraph covers every intermediate numerical phase; do not narrate each number.

For the square root of two, I use a tolerance. Watch the values settle. The maximum number of updates is a cap on work, not a request to compute every one. The lazy sequence is consumed only until the stopping test succeeds or that cap is reached.

Phase last

> End if needed. Pause on the stopping message.

Here the change falls below ten to the minus twelve. That is a bound on successive changes, not automatically a bound on the error in an arbitrary algorithm. Hitting the work cap would be a different reason to stop. The full helper is here for reference; the important interface is still these two supplied functions. In the examples that follow, both the state and the stopping test will change.

09 Plan tomorrow’s turbine settings

06:24–07:25 · 116 words · 6s visual holds

Phase 1

> Point to the forecast and the turbine scene.

Our first problem is a wind farm. A forecast tells us the available generation; we choose how much to retain at each turbine and hour. At noon, this example offers eighty-four kilowatt-hours per turbine. Setting one retains it all; one half retains forty-two. We share one illustrative generation forecast across all six turbines. We choose curtailment, not wind speed.

Phase 2

> Right. Point to the vector and then the whole farm.

Six turbines over twelve hours gives seventy-two settings, stored under the keyword x in a map. A colon marks a keyword: a label used as a key. As-plan pairs those vector entries with turbine-hour keys. It changes the representation, not the settings. Half everywhere is just an interior starting guess for the search, not a recommended policy.

10 What makes one plan better?

07:25–08:56 · 171 words · 9s visual holds

Phase 1

> Read the three whole-plan rows.

If electricity were our only concern, we would retain everything. Stopping turbines gives up electricity. It also reduces the exposure of birds passing the farm. We need to say how we compare those two goals. The cost function on the left adds up electricity lost across the whole plan. Clojure maps can be called as lookup functions: run applied to a turbine-hour key returns that setting. For produces the contributions; reduce plus adds them.

Phase 2

> Right. Point to T2 and T3 at noon.

The turbines are not equally exposed. Each cell here is exposure at full operation for a turbine and an hour. T3 is close to the flight path; T2 is farther away. The hourly traffic pattern is shared, and location scales it. So curtailing T3 at noon removes much more exposure for the same electricity loss. This is an illustrative exposure index, not a prediction of birds killed.

Phase 3

> Right. Return to the tradeoff table.

Neither column alone tells us which compromise to choose. First I'll show how we improve a plan for one numerical cost. Then we'll supply a cost that includes both concerns.

11 Optimization is iterate

08:56–10:51 · 221 words · 9s visual holds

Phase 1

> Point to the two sampled costs, then the returned fn.

Suppose we score only electricity lost. How do we know which way to move a setting? Hold seventy-one settings fixed, and nudge the remaining one a little below and above its current value. Compare the two costs. Their difference, divided by the change in the setting, estimates a slope. Here it is minus eighty-four. Increasing this setting reduces lost electricity.

Those slopes form the gradient vector. Gradient takes a cost function and returns a function that estimates that vector at any supplied plan. It samples numbers; there is no symbolic differentiation here.

Phase 2

> Right. Read one row, then gesture across all six.

Descent-step subtracts a small multiple of each slope. Half minus point zero zero zero one times minus eighty-four gives point five zero eight four. The argument destructuring extracts x from the state map. Assoc returns the updated map without changing the old one. Hash-parenthesis is a shorter way to write a lambda; percent denotes its argument. All seventy-two coordinates use the same old vector. The noon settings move equally here because their electricity forecasts are equal.

Phase 3

> Right. Follow solver-step from the inside outward.

Clamped takes that update function and returns another function. It calls the update, then clips every proposed setting to zero through one. The solver is now a function from a plan state to a new plan state. Iterate feeds each new state back. All the problem-specific work is inside the step we supplied.

12 Which plan should win?

10:51–12:41 · 209 words · 9s visual holds

Phase 1

> Point to A and B, then the electricity row.

Now let's choose the tradeoff. Plan A retains half everywhere. Plan B is identical except that T3 stops at noon. B gives up another forty-two kilowatt-hours. On electricity alone, A wins.

Phase 2

> Right. Explain the small risk-14 example before the table.

I choose a target of twelve for total exposure. Above that, I add one hundred times the squared excess to the cost. A risk of fourteen is two above target, so it adds four hundred. Twelve and one hundred are modelling choices. The weight converts squared exposure into score units comparable with the electricity-loss term.

Now B wins: its reduction in exposure is worth more under this chosen score than the extra electricity lost. Both candidate plans still exceed the target; we have compared a direction of improvement, not finished the optimization. This is a soft penalty. It encourages lower exposure; it does not guarantee a strict limit.

Phase 3

> Right. Point to cost, risk and the returned fn.

Here is the wrapper. It receives the original cost, a risk function, and the parameters, then returns a new cost function. Comp in the definition composes functions right to left: first turn the vector into a plan, then measure its risk. Both old and new costs accept the same vector and return one number. Gradient can sample either one. We change the objective without changing the update algorithm or iterate.

13 Same updates, different objectives

12:41–14:04 · 139 words · 16s visual holds

Phase 1

> Explain the grid before pressing Space.

Both sides start with the same plan, rate, and stopping test. A row is one turbine; a column is an hour; darker means more generation retained. The only difference is the supplied cost function.

Phase 2–penultimate

> Space to play; pause near the middle if useful. Speak over the changing grids. This paragraph covers every intermediate numerical phase; do not narrate each number.

On the left, electricity alone pushes the settings upward. On the right, the bird penalty changes which settings are worth increasing or reducing. The plan changes most where traffic and exposure are high. These frames are successive drafts of tomorrow's plan. The day itself is not passing yet.

Phase last

> End. Read the displayed final generation and risk, without memorizing them.

Compare the final generation and exposure. The penalty buys a reduction in exposure by giving up some generation. Here both traces stop because the largest setting change is below point zero zero one. The cap is six hundred updates. That says the numerical search has settled under our tolerance; it is not a general proof of optimality.

14 Play the resulting day

14:04–14:47 · 62 words · 14s visual holds

Phase 1

> Let the hour marker cross several columns.

Now we play a day through a fixed schedule. The red rectangle marks the current hour. First, every turbine retains all available generation.

Phase 2

> Right. Point between the current column and the turbines.

With the penalty, the chosen schedule curtails exposed turbines during busy hours. The clock is advancing; the plan is no longer changing. The optimizer produced the data being replayed. The moving birds illustrate that model, not measured field results.

15 Buy hydrogen before the customers arrive

14:47–16:53 · 236 words · 12s visual holds

Phase 1

> Point to tank capacity, supplier table, then customer probabilities.

For decision-making, consider a hydrogen station. It has room for four units, and each customer truck takes one. The station buys from a supplier before it knows how many customers will arrive. Regular weeks offer up to three units at a price of two. Tight weeks offer only one unit, at a price of six. Those weeks alternate. The tank's physical capacity stays four; the supplier's available quantity changes.

Phase 2

> Right. Follow the delivery into the tank.

Customer arrivals are uncertain: forty percent of weeks have no trucks, fifteen percent have one, and forty-five percent have a convoy of three. The average is one and a half, but the full distribution matters. We use the same distribution each week, independently.

Here we buy one unit in a regular week, bringing our stock to two. The order is chosen before we see which customer outcome happens.

Phase 3

> Right. Point to the turned-away truck.

A convoy arrives. We sell to two trucks and turn the third away. This is one possible outcome, not the demand we knew when choosing the order.

Phase 4

> Right. Point first to reward, then next-state.

We sell each unit for ten, so this outcome earns twenty minus the purchase cost of two: eighteen. Unsold units would cost one each to hold for the week. The next state contains leftover stock and the next supply window. Our objective is expected profit over repeated weeks, multiplying each following week's contribution by another factor of point nine. This is a modelling preference for earlier returns, not a ten-percent hydrogen leak.

16 Buy for this week—or the next?

16:53–17:45 · 96 words · 6s visual holds

Phase 1

> Read the state and compare the two order columns.

The state is a pair: supply week and stock. Orders returns feasible purchase quantities.

In a regular week with one unit already in stock, buying two covers the largest possible convoy this week. Buying three fills the tank to four. Why carry more than any current convoy needs? Look at the leftovers: either way, the next week has expensive, limited supply.

Phase 2

> Right. Pause for the tradeoff.

That extra unit may avoid an expensive future purchase, or it may sit unsold and incur holding cost. We need a value for each possible next state. That is what the Bellman calculation will build.

17 Future values change the order

17:45–19:49 · 235 words · 12s visual holds

Phase 1

> Keep the state fixed throughout all four reveals.

V is a table: for each supply week and stock level, it estimates profit from then onward. Start all entries at zero. Clojure maps are functions too, so V of a state is a lookup in that table. For each possible order, average over the three demand outcomes: this week's profit plus point nine times the value of the next state. With every future value zero, only this week matters. Order two wins.

Phase 2

> Right. Point to the unchanged best order.

After one update, the table contains optimal one-week values. Using those as the future term now scores a current order plus one more week. We score the same orders again, from the same initial situation. Order two still looks best. Nothing has been bought; this is calculation.

Phase 3

> Right. Identify the jump to update three.

We skip update two on this slide. After three updates, V accounts for three weeks; scoring an order with it looks one week further ahead. Now order three wins. The extra stock has value because it carries us into the tight supply week. We did not add an if statement saying to buy more before a tight week. The recommendation follows from the transitions, costs, and future values.

Phase 4

> Right. Point to fn, then max, then zipmap.

Bellman takes our model and returns an update function. For every state it scores feasible orders, takes the maximum, and uses zipmap to associate each state with its new value. There are ten states: two supply windows times five stock levels. Iterate applies this table-to-table function repeatedly.

18 Hydrogen: iterate the value table

19:49–21:27 · 174 words · 14s visual holds

Phase 1

> Identify the axes and colours before starting the animation.

Each pair of bars is a starting stock level. Blue is a regular week; amber is a tight week. Height is expected future profit in money units. It is not the amount of hydrogen. All bars start at zero because that was our initial guess for future value.

Phase 2–penultimate

> Space. Follow the stock-1 row; allow the animation to finish. This paragraph covers every intermediate numerical phase; do not narrate each number.

Every update revises all ten values using the old table. The order recommendation beside stock one changes as those future values improve. We're expanding how far ahead the calculation can account for consequences. The supplier cycle is already part of the state, so we keep using the same update function.

Phase last

> End. Point to the residual and the two stock-1 recommendations.

The largest change is now below our tolerance. In the stopping lambda, percent-one and percent-two stand for the two successive tables. With one unit in stock, the recommended action depends on the week: buy three when supply is cheap, and buy none in the tight week. For a fixed finite model and discount below one, exact value iteration has a convergence argument. Later I'll show the bound behind it; here we are watching a floating-point calculation.

19 The best order depends on the supply window

21:27–22:40 · 139 words · 6s visual holds

Phase 1

> Read stock 1 first; compare the strongest simple rule with Bellman.

This policy maps situations to orders. The comparison evaluates each rule from the same initial state over twenty discounted weeks, weighting all demand branches. Filling toward three is already good. Bellman does a little better; filling toward four does worse. This evaluates a stationary policy over twenty weeks, rather than optimizing a business that closes after week twenty.

Phase 2

> Right. Point to the empty-tank tight-week row.

There is a less obvious recommendation here. With an empty tank in a tight week, buy one, even though that purchase has negative expected profit this week. The cost is paid now. If nobody arrives, the unit carries into next week, when cheap supply is capped. That future benefit outweighs the immediate expected loss in this model. Looking only at immediate expected profit misses that value. This comes from the actual model; it is not a general claim about hydrogen markets.

20 Validate the turbine controller

22:40–24:46 · 239 words · 12s visual holds

Phase 1

> Return explicitly to the wind farm; read input, controller, output.

Back at the wind farm, we ask a different question. A confident bird detection must cause a STOP command in that modelled minute. Can any event sequence leave the command at RUN instead? Detection is an input. We are validating the controller's response, not a bird-recognition model.

Phase 2

> Right. Point to tick and detect branches.

Here is one controller. A timer makes a decision every ten minutes. A detection arriving between ticks is recorded, but the action is unchanged. That can leave the turbine commanded to run after a confident detection. We have a concrete rule and a concrete failure.

Phase 3

> Right. Explain the missing table row before reading the wrapper.

We can wrap the controller so a stronger detection triggers an immediate decision. But its decision table has another problem: an unfamiliar situation has no row. The default stopping probability is zero. The variable u is a sampled draw compared with that probability. Our finite exploration represents the relevant draw bands explicitly, rather than sampling during the search. Reacting sooner does not fix a bad decision. In this table, a confident detection in a known situation does stop the turbine.

Phase 4

> Right. Follow with-shield and compare the two computed outputs.

With-shield wraps the decision function. In known situations it delegates to that function. In an unknown situation, any detection produces STOP. The same event now gives STOP instead of RUN. The construction below shows the wiring: the shielded decision is supplied both to the timer controller and to the immediate-reaction wrapper. One example is encouraging. Next we search all the event sequences represented by this finite model.

21 Can any event sequence reach RUN + confident bird?

24:46–27:23 · 269 words · 27s visual holds

Phase 1

> Point to the rule predicate and the initial state count.

Our search state is now a set of controller states. The predicate at the bottom says what failure means: detection level two and action RUN. Keywords such as det and action select fields from a map. We start from the controller's possible initial states.

Phase 2

> Right. Explain the branch diagram and the set-expansion code together.

The next minute can contain no detection, a weak one, or a confident one. The diagram shows one branch; the counts cover the full exploration. Reach-step takes that successor function and returns a set update. Mapcat collects successors; into adds them to the states already known. A set removes duplicates. Detection and timer transitions are separate so a reset cannot hide a failure.

Phase 3

> Right. Pause on the timer-only failure count.

Repeat until adding successors changes nothing. For the timer-only controller, states violating the rule are reachable. The search has found a real problem in this model.

Phase 4

> Right. Identify the change of controller.

Now swap in the controller that reacts immediately. Keep the same search, event model, and failure predicate.

Phase 5

> Right. Point to the unseen-situation branch.

The unknown-context branch still leads to RUN after a confident detection. That missing table row survives the timing fix.

Phase 6

> Right. Pause on the second failure verdict.

The second controller still fails. We need both the immediate reaction and a safe decision for the unknown situation.

Phase 7

> Right. Identify the shielded controller.

Finally, supply the shielded controller to the same exploration.

Phase 8

> Right. Point to STOP on confident detection.

The branch that failed before now issues STOP.

The search continues, so we are checking more than that one branch.

Phase 9

> Right. Pause on set closure.

The set is now closed under every encoded successor, without a failure. That establishes the rule for this finite model and its initial states. Set equality is the stopping test. Finiteness and the growing set guarantee eventual closure; infinite-state systems need not terminate. This says nothing about sensor accuracy or physical braking.

22 iterate underneath all three

27:23–28:08 · 89 words · 3s visual holds

Phase 1

> Read across each row; do not repeat the whole story.

Underneath all three examples is iterate. What changes is what a state means. A vector of turbine settings becomes a value table, then a set of reachable states. The step changes with it, and so does the stopping test. Iterate exposes the intermediate states as a trace we can inspect. This is an implementation pattern, not a claim that the three algorithms have the same guarantees. Set closure finishes a finite search; small numerical movement needs a separate interpretation. Now let’s use the shared interface to change the update.

23 Change the pace

28:08–28:57 · 89 words · 6s visual holds

Phase 1

> Return to the electricity-only slope at turbine 1, noon.

Back to our gradient update. The step size is an argument to descent-step. Here the familiar setting moves from one half to point five zero eight four.

Phase 2

> Right. Point to the changed rate and result.

Double that argument and the proposed movement doubles. A carefully factored imperative program could expose the same parameter. The benefit here is that the update is already a value we can pass, replace, and wrap; no traversal code needs editing. This makes experiments easy, although a larger step can overshoot. It is control over the algorithm, not a guarantee of faster convergence.

24 Give the update momentum

28:57–29:51 · 100 words · 6s visual holds

Phase 1

> Point to the returned fn and the velocity in state.

Momentum remembers the previous movement. This wrapper calls the original step, measures its proposed change, and adds ninety percent of the previous velocity. The new velocity is carried in the next immutable state. The update is still a function from state to state, so iterate does not need to change.

Phase 2

> Right. Follow the wrappers inside out; read the measured table.

The base step performs descent; momentum wraps it; clamped wraps the result. Clamping clips position but keeps velocity, a particular boundary choice. These actual runs use the same cost, start, rate and stopping test. Momentum takes fewer updates here. That is not a general convergence guarantee or a wall-clock benchmark.

25 Observe each update

29:51–30:52 · 115 words · 6s visual holds

Phase 1

> Point to the return value after the observer call.

Logging is another wrapper. Call the original step, give the old and new states to an observer, then return the new state unchanged. The numerical update is still the same function; the observer is where we deliberately perform an effect.

Phase 2

> Right. Point to the atom and four logged rows.

This observer records costs in an atom, Clojure's explicit mutable reference. Taking five states includes the starting state, so it runs four updates and records four costs. The vector forces the lazy sequence to be realized. The starting state itself causes no log entry. Reusing the same realized sequence reuses its cached values; constructing and consuming a fresh sequence runs the effects again. We can observe the algorithm without weaving logging through its numerical code.

26 Rainbow

30:52–31:31 · 74 words · 3s visual holds

Phase 1

> Pause before moving from examples to proof.

Clojure is dynamically typed. Some of you prefer static types, which catch useful classes of mistakes. Proof lets us state a more specific property of an update and check a term that establishes it. This is available to users of static languages too.

Rainbow is a theorem prover written in Clojure. I'll show a small proof so you can read the notation, then return to a property of the Bellman operator we just used.

27 A proof is a function

31:31–32:51 · 154 words · 6s visual holds

Phase 1

> Follow the right-hand diagram, then match it to the left.

Suppose P, Q, and R are propositions. We have a function taking a proof of P to a proof of Q, another taking a proof of Q to a proof of R, and a proof of P. How do we produce R? Apply f, then g. It is function composition.

This is actual Rainbow term syntax. Forall introduces arbitrary propositions; the arrow states implications. Fn binds the proof arguments, and app applies them. The body is g applied to f applied to p. The universals are part of the statement: the term must work for arbitrary P, Q, and R. Rainbow checked it in an empty environment, with no axioms.

Phase 2

> Right. Point to the shortened, rejected term.

If I offer only f of p, I have produced Q, while the statement asks for R. The checker rejects it. Those two verdicts are recorded from the kernel. This is a small proof we can read completely, before looking at a larger theorem.

28 Bellman shrinks disagreement

32:51–33:49 · 110 words · 6s visual holds

Phase 1

> Point back to the hydrogen value table; define the gap.

Take two estimates of future profit, V and W. Call their largest difference across all states epsilon. That is the worst disagreement between corresponding entries, not an average. For our discounted Bellman operator, one update makes that difference at most gamma times epsilon. Gamma is point nine in the station model. The claim needs the same model, valid probabilities, and a nonempty set of feasible actions.

Phase 2

> Right. Point to the computed 10 to 9 example.

For illustration, start one table at zero everywhere and the other at ten. Applying the browser's Bellman function to both reduces the gap from ten to nine. That is one example. A proof has to cover every pair of value tables satisfying the assumptions.

29 The proof follows the code

33:49–35:21 · 166 words · 12s visual holds

Phase 1

> Fix a state and one order before comparing the two calculations.

Pick one state, one order and one demand outcome. Compare the score using V with the score using W. The immediate reward is identical, so it cancels. Only the estimate of future value differs, by at most epsilon.

Phase 2

> Right. Point to normalized probabilities.

Multiply by the discount and average over outcomes. Probabilities are nonnegative and sum to one, so the bound becomes gamma times epsilon. This is the weighted reduce in the code.

Phase 3

> Right. Pause here; this is the mathematical jump.

Now take the best action. Take the order that wins using V. Its score can beat that same order under W by at most our bound. The best order under W is at least as good as that particular order. So taking a maximum cannot enlarge the bound. The maximizing actions do not have to be the same.

Phase 4

> Right. Point to the absolute value.

Swap V and W to get the reverse inequality. Together they bound the absolute difference for every state. The argument follows the structure of the operator: outcomes, weighted sum, maximum. That makes the proof easier to relate to the code.

30 What the kernel checked

35:21–36:18 · 113 words · 3s visual holds

Phase 1

> Point to the checked theorem, then its scope.

Rainbow checked an exact arithmetic version of this bound, with no axioms in its dependency closure. This receipt names that theorem. It does not certify the floating-point JavaScript implementation running these slides. Connecting that program to the mathematical operator would require a separate argument.

There is a useful mathematical consequence. For an exact candidate value table, measure its residual: the difference from one more Bellman update. With discount point nine, an exact residual below point zero zero one bounds its error from the fixed point by point zero one. The shrinking gaps form a geometric series. That explains what a stopping tolerance can mean; it is not a certificate for our floating-point run.

31 Applied Higher-Order Functions

36:18–36:58 · 76 words · 3s visual holds

Phase 1

> Gesture to the reserved book image; leave the URL visible.

Today I concentrated on iterate, but you've also seen map, reduce, function composition, and functions that return other functions. My book, Applied Higher-Order Functions, develops a chosen toolkit of about twelve higher-order functions and their composition. That is the book's organizing choice, not a claim that all of operations research reduces to twelve functions. If you want to build more of these algorithms and work through the examples, that is where I've put the longer treatment.

32 Thank you

36:58–37:09 · 18 words · 3s visual holds

Phase 1

> Leave the contact details visible for questions.

Thank you. You can reach me at jiri at flexiana dot com. I’d be happy to take questions.